

MSc in High Performance Computing

Coursework for Threaded Programming Part 2

The object of this assessment is to implement an alternative loop scheduling algorithm in OpenMP.

You are provided with a piece of code which contains two loops which have been parallelised with OpenMP directives. Instead of using the work sharing loop directive, the loop is scheduled “by hand” using a parallel region. The implementation you are provided with corresponds to the `STATIC` schedule kind. The code measures the execution time for 1000 repetitions of each loop, and includes a verification test for each loop.

The code can be found on the course pages on Learn. You may choose to work with *either* the C (`loops2.c`) *or* Fortran 90 (`loops2.f90`) version.

You should always compile the code with the `-O3` option to ensure a high level of sequential optimisation, but you must *not* alter the routines which contain the body of the parallel loops.

We will use the term *chunk* in the same sense as in the OpenMP standard, i.e. a contiguous, non-empty subset of the iterations of a loop.

Affinity scheduling

Affinity scheduling can be described as follows:

- Each thread is initially assigned a (contiguous) *local set* of iterations.
- For a loop with n iterations, and p threads, each thread’s local set is initialised with n/p iterations. (If p does not divide n exactly, choose a suitable distribution of the extra iterations.)
- Every thread executes chunks of iterations whose size is a fraction $1/p$ of the *remaining* iterations its local set, until there are no more iterations left in its local set.
- If a thread has finished the iterations in its local set, it determines the thread which has most remaining iterations (the “most loaded” thread) and executes a chunk of iterations whose size is a fraction $1/p$ of the remaining iterations in the “most loaded” thread’s local set.
- Threads which have finished the iterations in their own local set repeat the previous step, until there are no more iterations remaining in any thread’s local set.

You should take great care with the implementation of this algorithm to ensure that threads are correctly synchronised.

Once you have implemented the algorithm, run your code on 1, 2, 4, 6, 8, 12 and 16 threads and compare the results to the best built-in OpenMP schedule which you determined in Part 1 of the coursework.

Submission

You are required to submit the following:

1. A written report.
(Guideline length: 8-12 pages including figures and tables.)
2. Source code.

The deadline for both report and source code is 12.00pm Friday 2nd December 2016.

Your report should contain the following sections:

- a *short* introduction;
- a discussion of your implementation of the affinity scheduling algorithm, with particular attention to the data structures used and how the threads are synchronised. (Note that this section must be comprehensible without reference to the source code);
- a discussion of the results of running the affinity scheduling algorithm, including appropriate graphs and/or figures;
- some *brief* conclusions.

Your report should *not* contain any background material.

The code will be marked on design and readability as well as correctness and performance.

The maximum available mark for this assessment is **70**. Marks will be allocated as follows:

- Report content and presentation out of **30**.
- Source code out of **40**.